Movie Recommendation System using Big Data Analytics

Midterm Evaluation Report

Subject: Big Data

Submitted by

Krishanu Anand (23CSU166)
Yathansh Jhakar (23CSU352)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING,

SCHOOL OF ENGINEERING AND TECHNOLOGY

THE NORTHCAP UNIVERSITY

GURUGRAM-122017

Index

Abstract 3
1. Problem Definition 4
Objectives 4
Target users & expected outcomes 4
2. Dataset 5
3. Architecture & Technologies 6
4. Methodology 7
4.1 Ingestion & Bronze layer 7
4.2 Silver layer – cleaning 7
4.3 Gold layer – analytics 7
4.4 Recommendation model – Spark MLlib ALS 8
5. Results & Analysis 9
5.1 Exploratory analytics 9
5.2 Model evaluation 11
5.3 Sample recommendations 12
6. Dashboard 14
7. Cloud execution & orchestration 21
Cloud execution evidence 21
8. Conclusion & Future Work 24
References 25

List of Figures

Figure 1: Rating volume per year 9
Figure 2: Distribution of ratings 9
Figure 3: Genre popularity over time 10
Figure 4: ALS hyper-parameter search 11
Figure 5: Predicted vs actual ratings on the test split 12
Figure 6: Dashboard – Overview page 15
Figure 7: Dashboard – Content page 17
Figure 8: Dashboard – Audience page 17
Figure 9: Dashboard – Recommendations page 18

Figure 10: Dashboard – Model quality page.....19
Figure 11: Dashboard – Data pipeline page.....20
Figure 12: Google Cloud console – Dataproc jobs history.....22
Figure 13: Google Cloud console – Dataproc ALS job details 22
Figure 14: Google Cloud console – GCS bucket medallion layers 22
Figure 15: Google Cloud console – Dataproc cluster monitoring.....23
Figure 16: Google Cloud console – Billing free trial credits.....23

Abstract

This project builds an end-to-end Big Data pipeline on the MovieLens ml-25m dataset (25,000,095 ratings from 162,541 users on 59,047 movies). Data is ingested into Google Cloud Storage and processed with PySpark on Google Cloud Dataproc using a Bronze → Silver → Gold medallion architecture. Spark SQL and window functions produce analysis-ready tables, and a Spark MLlib ALS collaborative-filtering model predicts ratings with a test RMSE of 0.7874 (vs. 1.0578 for a global-mean baseline) and generates a personalised Top-10 list for every user. Results are presented in an interactive Streamlit dashboard, and the workflow is orchestrated with a runner script and an Airflow DAG.

Source code: https://github.com/anand-krishanu/AttractorField

1. Problem Definition

Streaming catalogues contain tens of thousands of titles; users struggle to discover movies they will enjoy, and content teams lack visibility into how tastes evolve. Manually curated lists do not scale to millions of users.

Objectives

- Ingest and store a large real-world ratings dataset in the cloud with full lineage metadata.
- Clean and validate the data (nulls, duplicates, invalid values, orphan keys, inconsistent text).
- Produce analysis-ready Gold tables: popularity, weighted ratings, genre trends, user segments, KPIs.
- Train and evaluate a scalable recommender (ALS) that beats a naive baseline.
- Deliver personalised Top-10 recommendations and an interactive dashboard built on real outputs.

Target users & expected outcomes

- Viewers: personalised, unseen movie recommendations and 'similar movies'.
- Product/content teams: KPIs, genre trends and user segments to guide acquisition and promotion.
- Outcome: an automated, reproducible pipeline from raw data source to end-user dashboard.

2. Dataset

MovieLens 25M is a stable benchmark dataset released by GroupLens Research (University of Minnesota) in 2019. It contains explicit 0.5–5 star ratings collected on movielens.org between 1995 and 2019; each user has rated at least 20 movies. Files: ratings.csv (userId, movieId, rating, timestamp), movies.csv (title with year, pipe-separated genres), tags.csv (free-text user tags) and links.csv (IMDb/TMDb ids). Source: https://grouplens.org/datasets/movielens/25m/ . The smaller ml-latest-small release (100k ratings) was used for local development and testing.

Citation: F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. ACM Transactions on Interactive Intelligent Systems 5, 4, Article 19. https://doi.org/10.1145/2827872

3. Architecture & Technologies

GroupLens → ingest.py → GCS raw/ → Bronze (Parquet) → Silver (clean, partitioned) → Gold (analytics) + MLlib ALS (output/) → Streamlit dashboard. Orchestration: run_pipeline.py (local Docker), scripts/dataproc_run.ps1 (Google Cloud) and an Airflow DAG (dags/movie_reco_dag.py).

Layer	**Technology**
Storage	Google Cloud Storage (data lake), Parquet (columnar, partitioned)
Processing	Apache Spark 3.5 / PySpark, Spark SQL, window functions
Machine learning	Spark MLlib ALS, RegressionEvaluator, RankingEvaluator
Cloud compute	Google Cloud Dataproc 2.2 (1 × e2-standard-2 master, 2 × e2-highmem-4 workers)
Orchestration	Apache Airflow DAG (Dataproc operators), PowerShell/Python runners
Visualisation	Streamlit + Plotly dashboard
Dev environment	Docker image apache/spark 3.5.3 (Java 17) mirroring Dataproc; GitHub with PR workflow

4. Methodology

4.1 Ingestion & Bronze layer

ingest.py downloads the official zip, writes a manifest (source URL, timestamp, file sizes) and uploads the raw CSVs to GCS — on Dataproc this runs on the cluster so data never passes through the client. bronze.py reads the CSVs with explicit schemas (no inference) and adds ingest_ts, source_file and dataset_version columns before writing Parquet, preserving the raw values for lineage and reprocessing.

4.2 Silver layer – cleaning

silver.py removes null keys, out-of-range ratings and invalid timestamps; resolves duplicate (user, movie) ratings by keeping the latest one with a row_number() window; drops ratings that reference unknown movies (left-semi join); converts epoch seconds to timestamps; extracts the release year from the title with a regex; turns the pipe-separated genres into arrays (mapping '(no genres listed)' to null); lower-cases and de-duplicates tags; and joins IMDb/TMDb links. Ratings are written partitioned by rating_year. Each check is logged:

Table	Check	Rows before	Rows after	Removed
movies	null id/title	62,423	62,423	0
movies	duplicate movieId	62,423	62,423	0
ratings	null fields	25,000,095	25,000,095	0
ratings	invalid rating/timestamp	25,000,095	25,000,095	0
ratings	duplicate user-movie	25,000,095	25,000,095	0
ratings	orphan movieId	25,000,095	25,000,095	0
tags	null/empty tag	1,093,360	1,093,360	0
tags	duplicate tag	1,093,360	1,093,359	1

4.3 Gold layer – analytics

- movie_stats: count, mean, std-dev and an IMDb-style Bayesian weighted rating WR = v/(v+m)·R + m/(v+m)·C (m = 90th percentile of vote counts) so that low-count movies do not dominate.
- top_movies_by_genre: rank() over a window partitioned by genre (top 20 per genre).
- genre_trends: Spark SQL over a temp view with LAG() to compute year-over-year growth per genre.
- user_activity: per-user volume, average rating, first/last activity and a segment (casual→power).

- user_top_rated, rating_distribution, top_tags and yearly_kpis (with a running cumulative total).

4.4 Recommendation model – Spark MLlib ALS

Alternating Least Squares factorises the sparse user × movie rating matrix into low-rank user and item factor matrices; predicted rating = dot product of the two factor vectors. A first run on all movies showed that titles with only 2–10 ratings received extreme factors and flooded every user's top-10 (precision@10 = 0), so the model is trained only on movies with at least 50 ratings (13,176 movies, still the vast majority of all ratings). Data was split 70/10/20 (train/validation/test, seed 42). A grid over rank ∈ {10, 20} and regParam ∈ {0.05, 0.1} (maxIter 10, non-negative factors, coldStartStrategy=drop) was evaluated on the validation split; the best configuration was refit on train+val and evaluated once on the held-out test split. Ranking quality was measured with precision@10 and NDCG@10 on a sample of 2,000 users, treating test movies rated ≥ 4 as relevant. Recommendations exclude movies the user has already rated. Item-item similarity was computed as cosine similarity between ALS item factors using Spark SQL higher-order functions.

5. Results & Analysis

5.1 Exploratory analytics

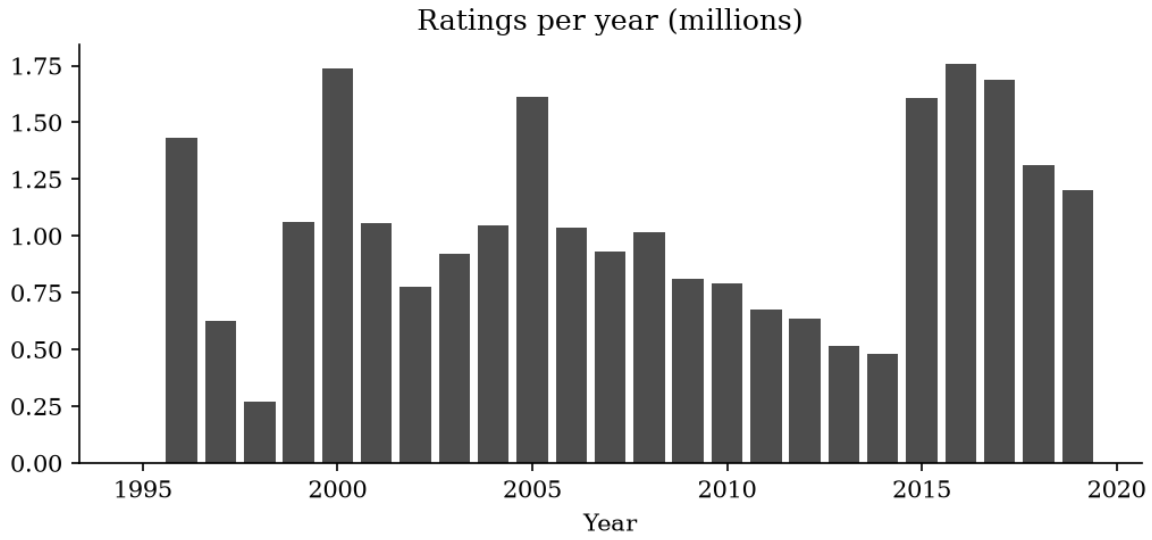


Figure 1: Rating volume per year

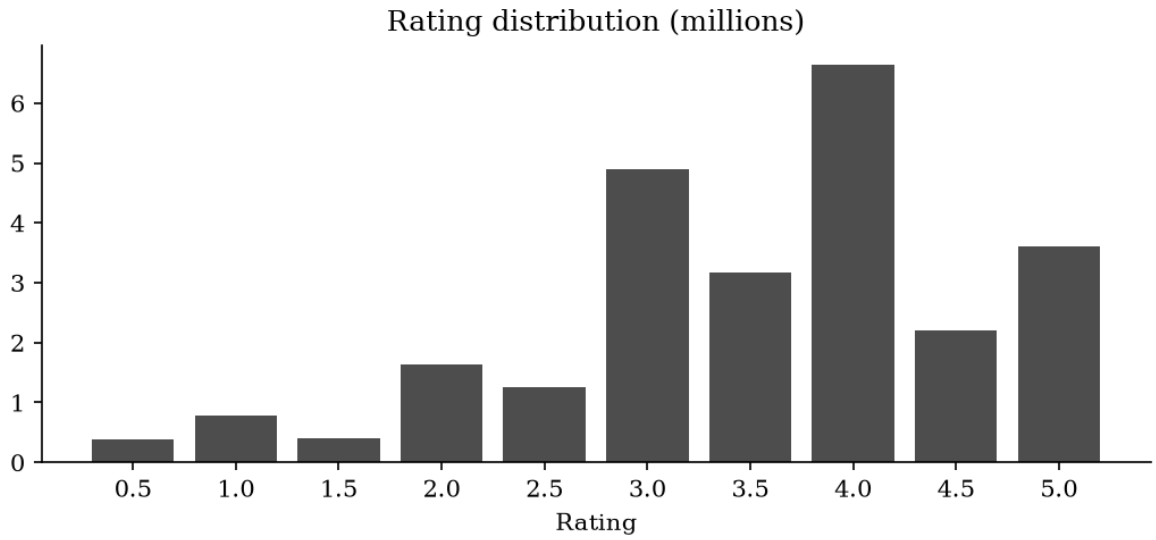


Figure 2: Distribution of ratings

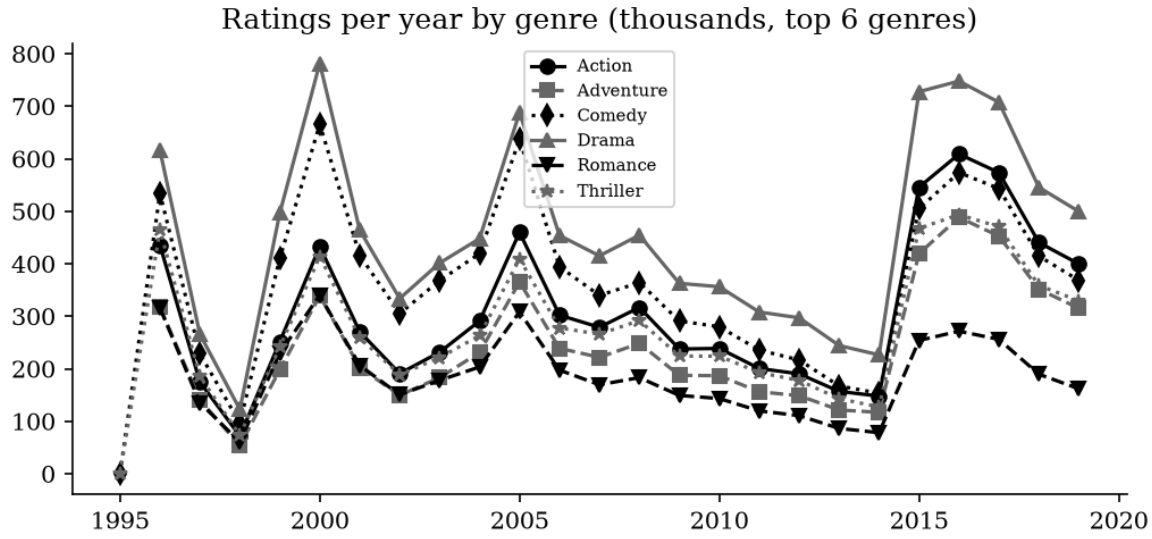


Figure 3: Genre popularity over time

Ratings are skewed positive: 50% of ratings are 4 stars or more and whole-star ratings dominate half-stars. The busiest year was 2016. Users rated on average 154 movies (median 71), showing a long-tail activity distribution; 6.0% of users are 'power' users with 500+ ratings.

Top-rated movies (Drama, by weighted rating)

Rank	Title	Year	Ratings	Avg rating	Weighted rating
1	Shawshank Redemption, The	1994	81,482	4.414	4.410
2	Godfather, The	1972	52,498	4.324	4.319
3	Godfather: Part II, The	1974	34,188	4.262	4.255
4	Schindler's List	1993	60,411	4.248	4.244
5	Seven Samurai (Shichinin no samurai)	1954	13,367	4.255	4.237
6	12 Angry Men	1957	16,569	4.243	4.229
7	Band of Brothers	2001	1,356	4.399	4.226

8	Fight Club	1999	58,773	4.228	4.224
9	One Flew Over the Cuckoo's Nest	1975	36,058	4.219	4.213
10	Casablanca	1942	26,890	4.207	4.199

5.2 Model evaluation

Rank	**regParam**	**validation RMSE**
10	0.050	0.800
10	0.100	0.814
20	0.050	0.791
20	0.100	0.811

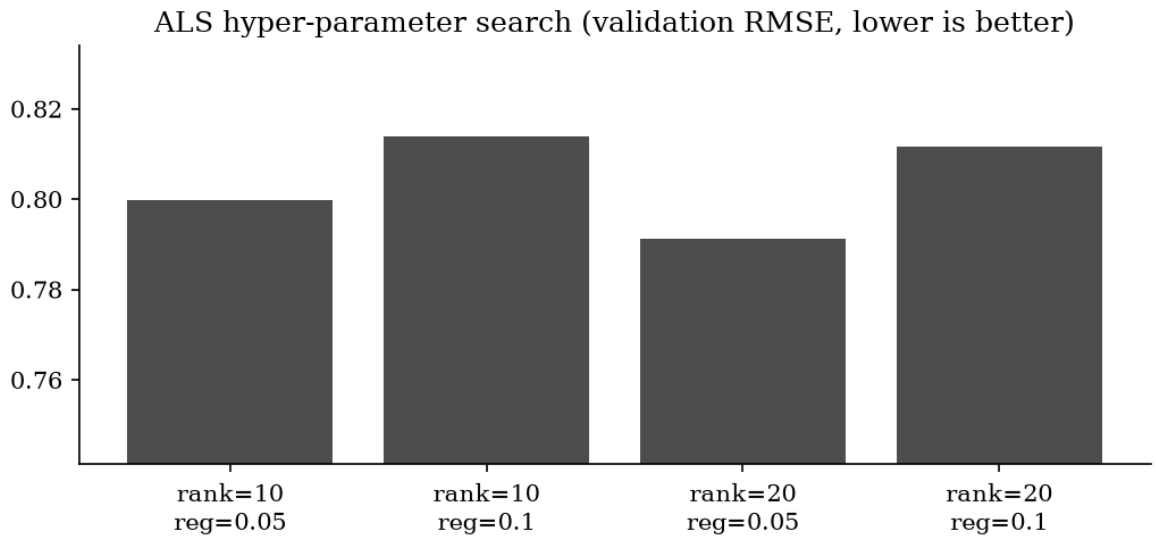


Figure 4: ALS hyper-parameter search

Metric	**Value**
Best parameters	rank=20, regParam=0.05
Train / Val / Test rows	17,252,352 / 2,462,699 / 4,929,877
Test RMSE (ALS)	0.787
Test RMSE (global-mean baseline)	1.058
Test MAE	0.609

Precision@10	0.024
NDCG@10	0.031

ALS reduces RMSE by 25.6% relative to always predicting the global mean, i.e. on average its predictions are within ~0.61 stars of the true rating. Ranking metrics are low in absolute terms, which is expected for top-10 retrieval from tens of thousands of movies when only held-out test ratings count as relevant; they serve as a baseline for future improvements.

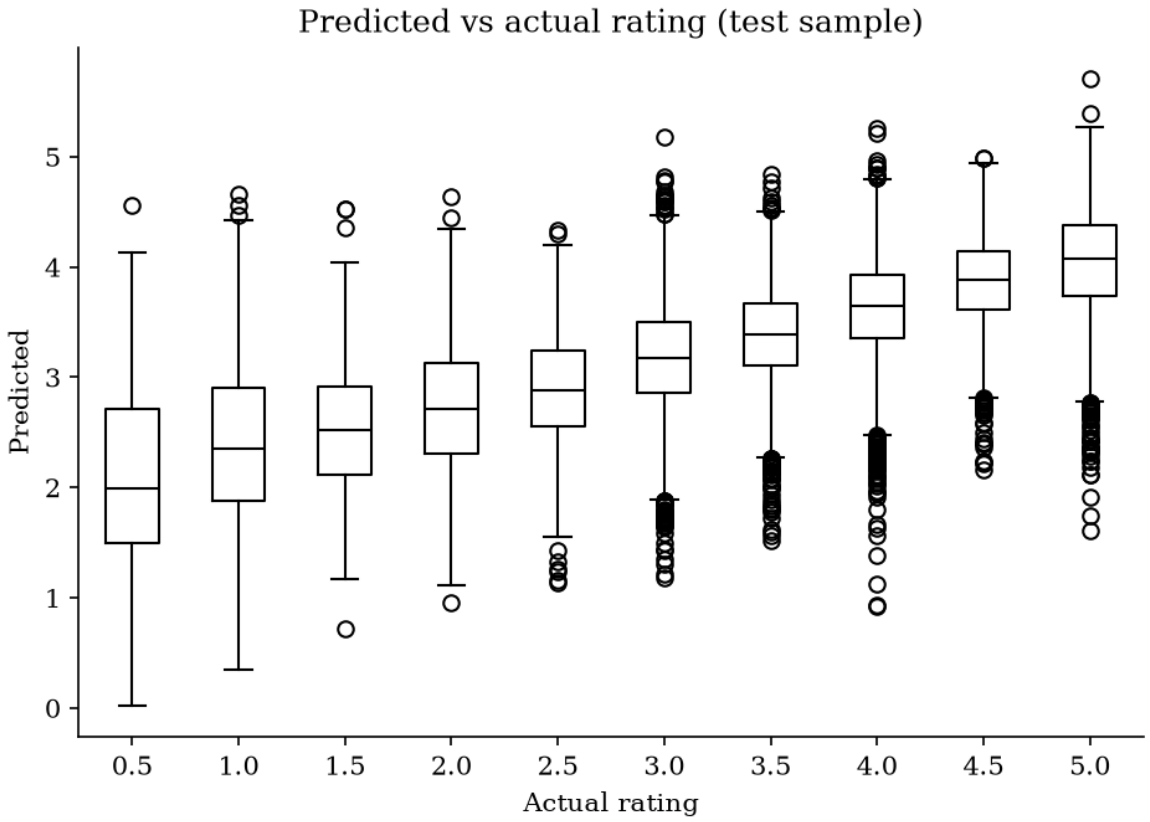


Figure 5: Predicted vs actual ratings on the test split

5.3 Sample recommendations

User 148 – favourite movies (highest rated):

Title	**Rating**
Paths of Glory	5.000
Good, the Bad and the Ugly, The (Buono, il brutto, il cattivo, Il)	5.000
Chinatown	5.000
Life Is Beautiful (La Vita è bella)	5.000
Seven Samurai (Shichinin no samurai)	5.000

User 148 – Top-10 recommendations (unseen):

Rank	**Title**	**Year**	**Predicted rating**
1	Planet Earth II	2016	4.540
2	Planet Earth	2006	4.533
3	Band of Brothers	2001	4.487
4	In the blue sea, in the white foam.	1984	4.475
5	Life	2009	4.437
6	Cosmos	–	4.426
7	The Adventures of Sherlock Holmes and Dr. Watson: Bloody Signature	1979	4.422
8	Alone in the Wilderness	2004	4.403
9	Blue Planet II	2017	4.310
10	Adventures Of Sherlock Holmes And Dr. Watson: The Twentieth Century Approaches	1986	4.298

6. Dashboard

The Streamlit dashboard (dashboard/app.py) reads the Gold and model output Parquet files produced by the Dataproc run. It was designed from business requirements: each page answers the question of one stakeholder group, and every page follows the same reading order of headline numbers, then the main chart, then detail. Chart titles state the finding rather than the metric, one accent colour marks what matters, numbers are abbreviated for readability, and labels use plain language. Run: streamlit run dashboard/app.py

Page	**Audience**	**What it answers**
Overview	Leadership	Platform health and recommender impact
Content	Content and acquisition	Top movies per genre, genre volume vs rating, trends, tags, similar movies
Audience	Growth and marketing	Engagement segments, active users over time, rating behaviour
Recommendations	Personalisation	A user's favourites next to their predicted Top-10
Model quality	Data science	RMSE vs baseline, tuning results, error analysis
Data pipeline	Data engineering	Medallion layers, data-quality checks, Dataproc job runtimes

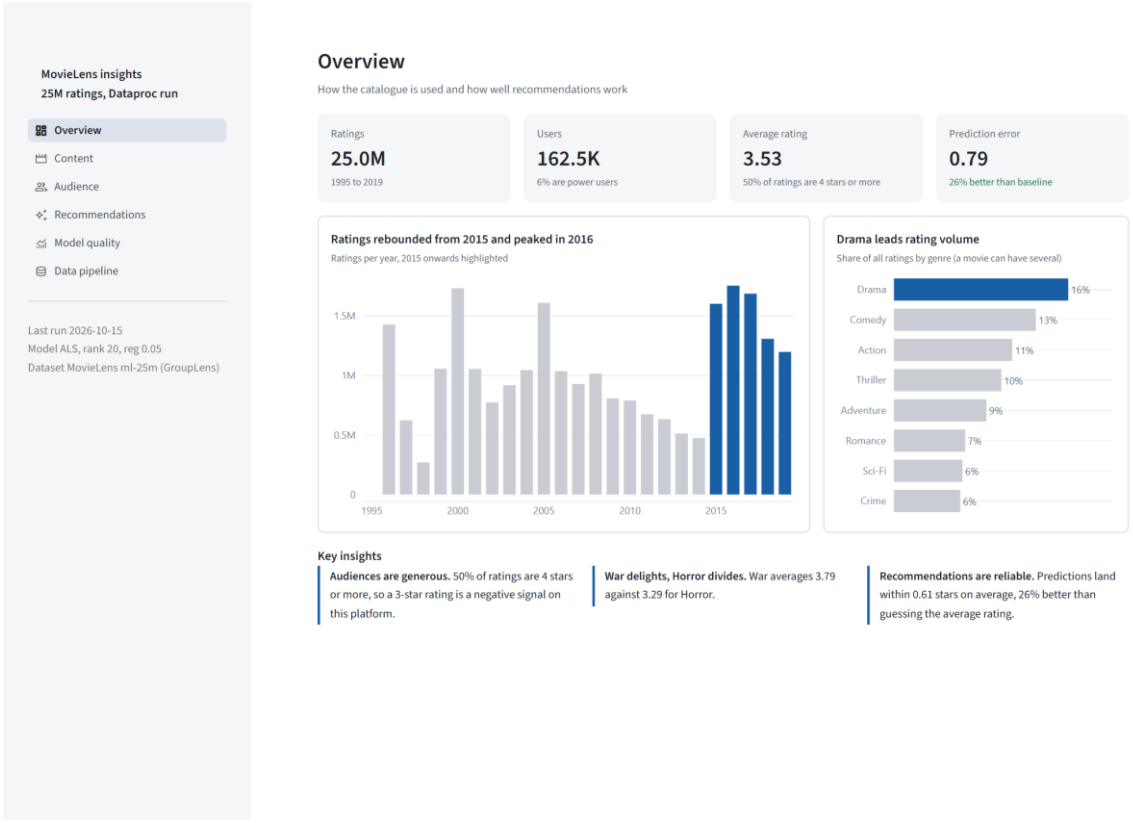


Figure 6: Dashboard – Overview page

MovieLens insights
25M ratings, Dataproc run

- Overview
- Content
- Audience
- Recommendations
- Model quality
- Data pipeline

Last run 2026-10-15
Model ALS, rank 20, reg 0.05
Dataset MovieLens ml-25m (GroupLens)

Content

Which genres and titles drive engagement, and what's growing

Genre: Drama

Ratings in genre	Average rating	Movies with 50+ ratings	3-year volume trend
11.0M	**3.68**	**6,288**	**-25%**
16% of all ratings	rank 5 of 19 genres	eligible for recommendations	2015 to 2018

Best Drama movies

Ranked by Bayesian weighted rating, which discounts films with few ratings

#	Title	Year	Ratings	Weighted
1	Shawshank Redemption, The	1994	81,482	4.41
2	Godfather, The	1972	52,498	4.32
3	Godfather: Part II, The	1974	34,188	4.25
4	Schindler's List	1993	60,411	4.24
5	Seven Samurai (Shichinin no samurai)	1954	13,367	4.24
6	12 Angry Men	1957	16,569	4.23
7	Band of Brothers	2001	1,356	4.23
8	Fight Club	1999	58,773	4.22
9	One Flew Over the Cuckoo's Nest	1975	36,058	4.21
10	Casablanca	1942	26,890	4.20

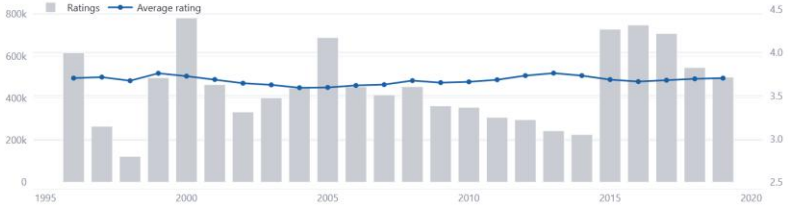
War rates highest, but volume lives elsewhere

Total ratings (log scale) vs average rating, by genre



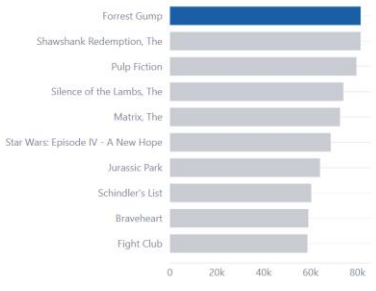
Drama activity over time

Ratings per year and average rating



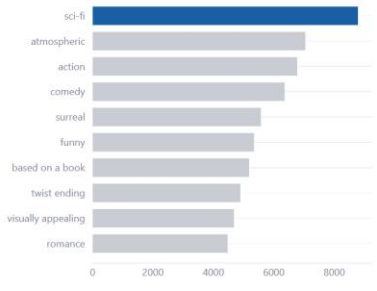
Forrest Gump is the most-rated title

Top 10 titles by number of ratings



"sci-fi" is the most used tag

Most used user tags



Find similar movies

Nearest neighbours by cosine similarity of ALS movie factors

Movie: Matrix, The

#	Similar movie	Similarity
1	Matrix Reloaded, The	0.88
2	Léon: The Professional (a.k.a. The Professional) (Léon)	0.87
3	Appleseed (Appurushîdo)	0.87
4	Usual Suspects, The	0.87
5	Fight Club	0.86
6	Twelve Monkeys (a.k.a. 12 Monkeys)	0.86
7	Minority Report	0.86
8	Ninja Scroll (Jûbei ninpûchô)	0.86
9	Animatrix, The	0.86
10	Professional, The (Le professionnel)	0.85

Figure 7: Dashboard – Content page

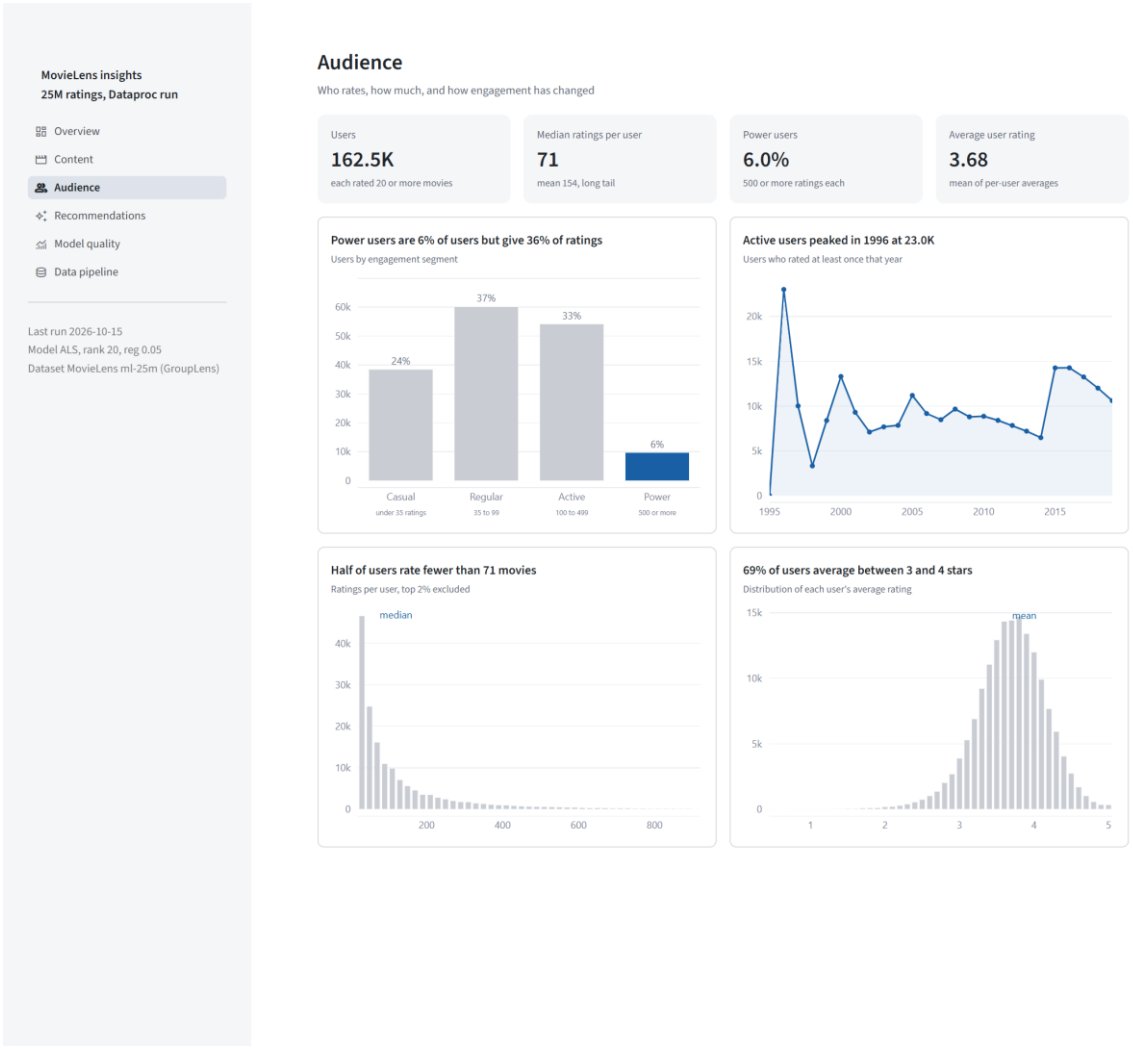


Figure 8: Dashboard – Audience page

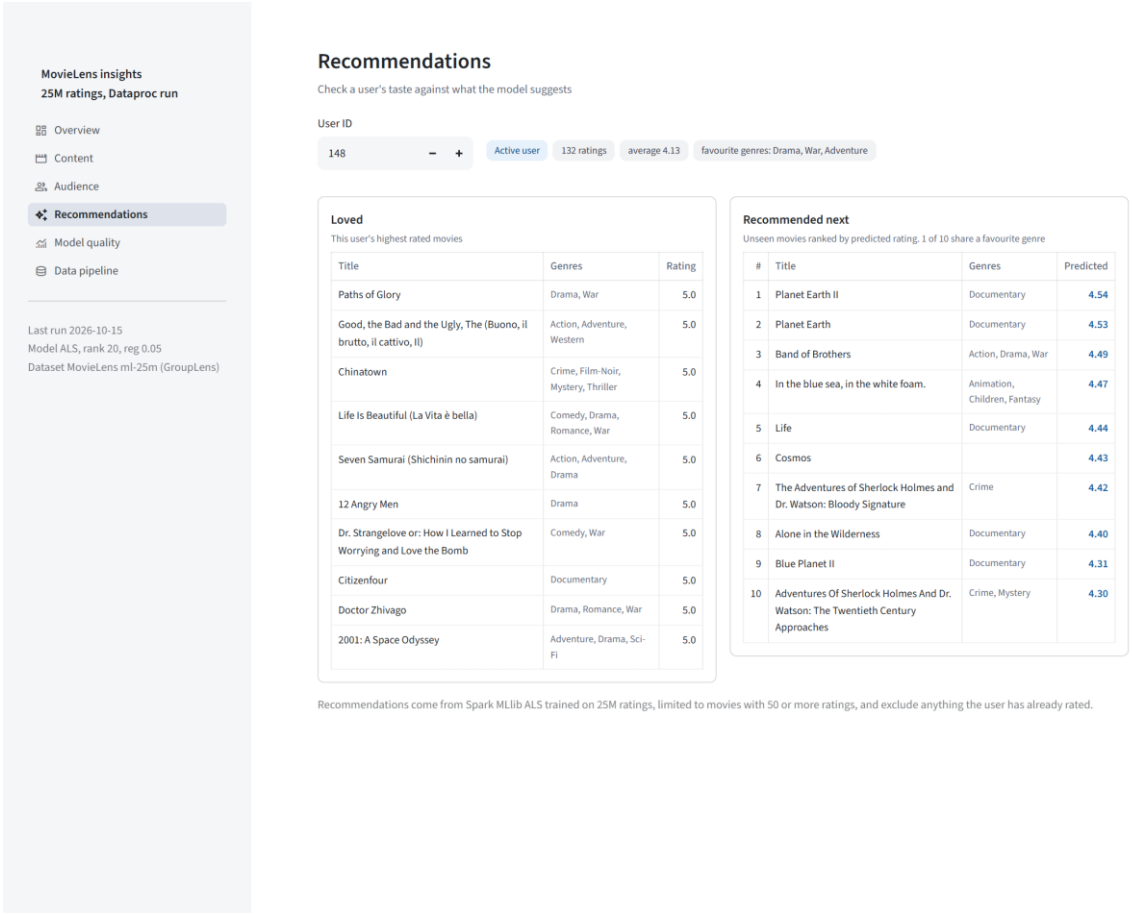


Figure 9: Dashboard – Recommendations page

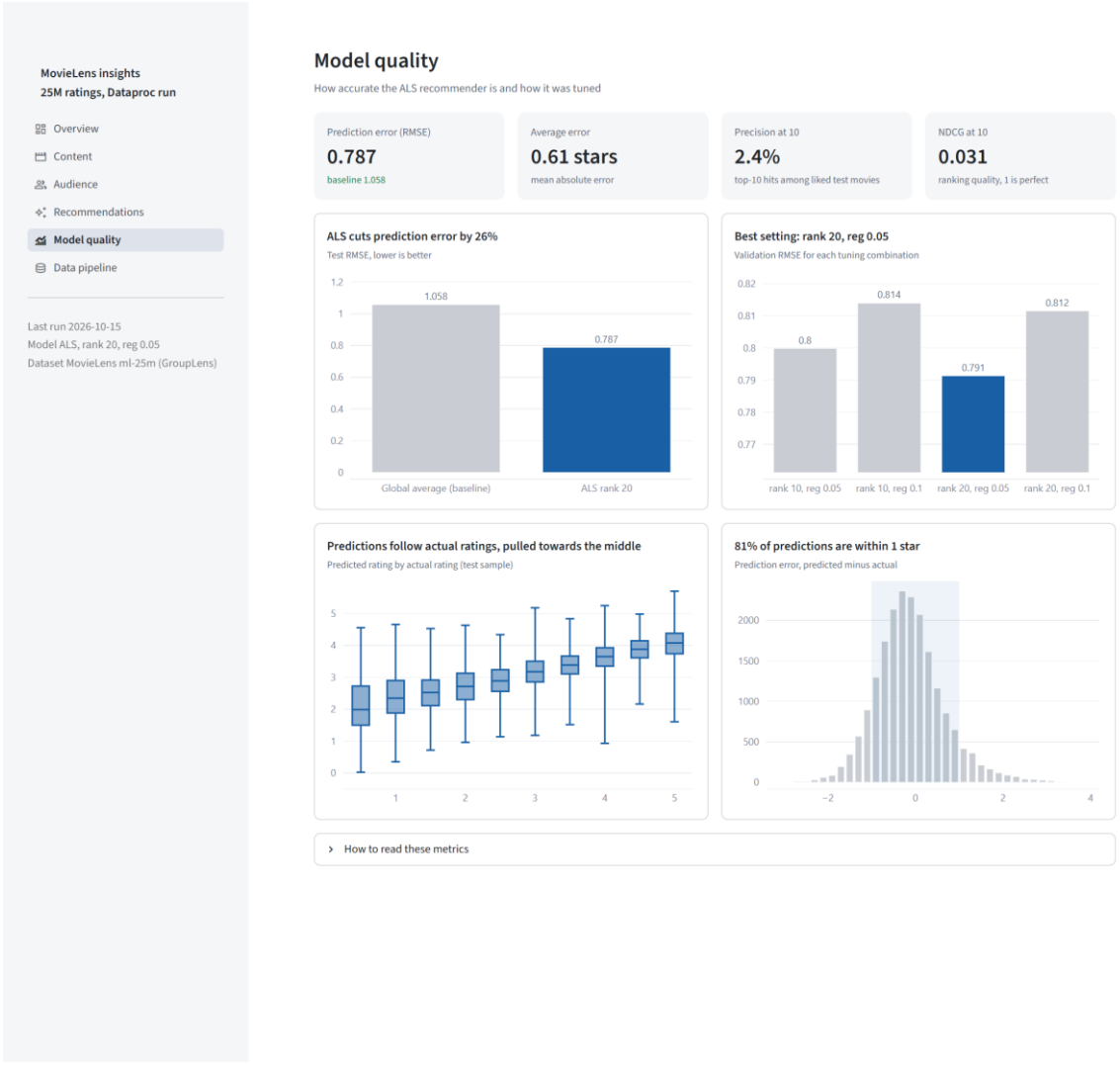


Figure 10: Dashboard – Model quality page

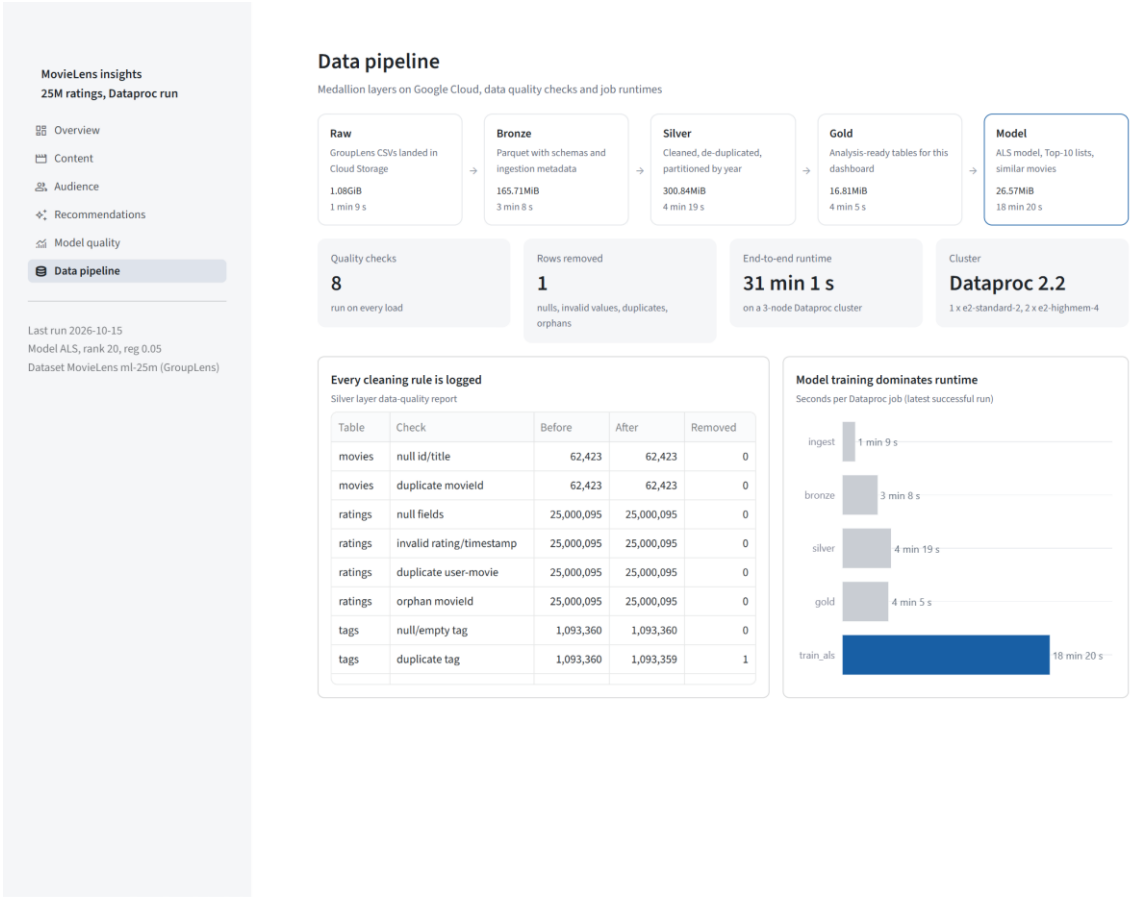


Figure 11: Dashboard – Data pipeline page

7. Cloud execution & orchestration

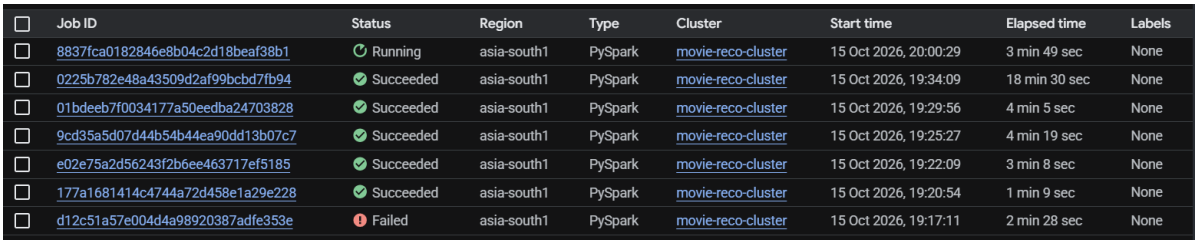
scripts/dataproc_run.ps1 creates a GCS bucket and a Dataproc 2.2 cluster (auto-deleted after 30 minutes idle), uploads the code, submits ingest, bronze, silver, gold and train_als as PySpark jobs, downloads the outputs for the dashboard and deletes the cluster to minimise cost. The same steps are defined as an Airflow DAG (create_cluster → ingest → bronze → silver → gold → train_als → delete_cluster, weekly schedule, retries, cluster deletion with trigger_rule=ALL_DONE) for deployment on Cloud Composer.

Cloud execution evidence

The repository folder docs/cloud_run_evidence/ contains the Dataproc job history (jobs.txt), the cleaned run logs, the GCS bucket listing per layer (bucket_listing.txt), the Spark driver output of the final ALS job and GCP console screenshots. Job history (project movie-reco-bigdata-2026, region asia-south1):

Job id	Script	State	Start (UTC)	Seconds
d12c51a5	ingest.py	ERROR	2026-10-15 13:47:11	148
177a1681	ingest.py	DONE	2026-10-15 13:50:54	69
e02e75a2	bronze.py	DONE	2026-10-15 13:52:09	188
9cd35a5d	silver.py	DONE	2026-10-15 13:55:27	259
01bdeeb7	gold.py	DONE	2026-10-15 13:59:56	245
0225b782	train_als.py	DONE	2026-10-15 14:04:09	1110
8837fca0	train_als.py	DONE	2026-10-15 14:30:29	1100

GCS layer	Size
raw	1.08GiB
bronze	165.71MiB
silver	300.84MiB
gold	16.81MiB
output	26.57MiB
code	18.56kiB



Job ID	Status	Region	Type	Cluster	Start time	Elapsed time	Labels
8837fca0182846e8b04c2d18beaf38b1	Running	asia-south1	PySpark	movie-reco-cluster	15 Oct 2026, 20:00:29	3 min 49 sec	None
0225b782e48a43509d2af99bcbd7fb94	Succeeded	asia-south1	PySpark	movie-reco-cluster	15 Oct 2026, 19:34:09	18 min 30 sec	None
01bdeeb7f0034177a50eedba24703828	Succeeded	asia-south1	PySpark	movie-reco-cluster	15 Oct 2026, 19:29:56	4 min 5 sec	None
9cd35a5d07d44b54b44ea90dd13b07c7	Succeeded	asia-south1	PySpark	movie-reco-cluster	15 Oct 2026, 19:25:27	4 min 19 sec	None
e02e75a2d56243f2b6ee463717ef5185	Succeeded	asia-south1	PySpark	movie-reco-cluster	15 Oct 2026, 19:22:09	3 min 8 sec	None
177a1681414c4744a72d458e1a29e228	Succeeded	asia-south1	PySpark	movie-reco-cluster	15 Oct 2026, 19:20:54	1 min 9 sec	None
d12c51a57e004d4a98920387adfe353e	Failed	asia-south1	PySpark	movie-reco-cluster	15 Oct 2026, 19:17:11	2 min 28 sec	None

Figure 12: Google Cloud console – Dataproc jobs history



Figure 13: Google Cloud console – Dataproc ALS job details

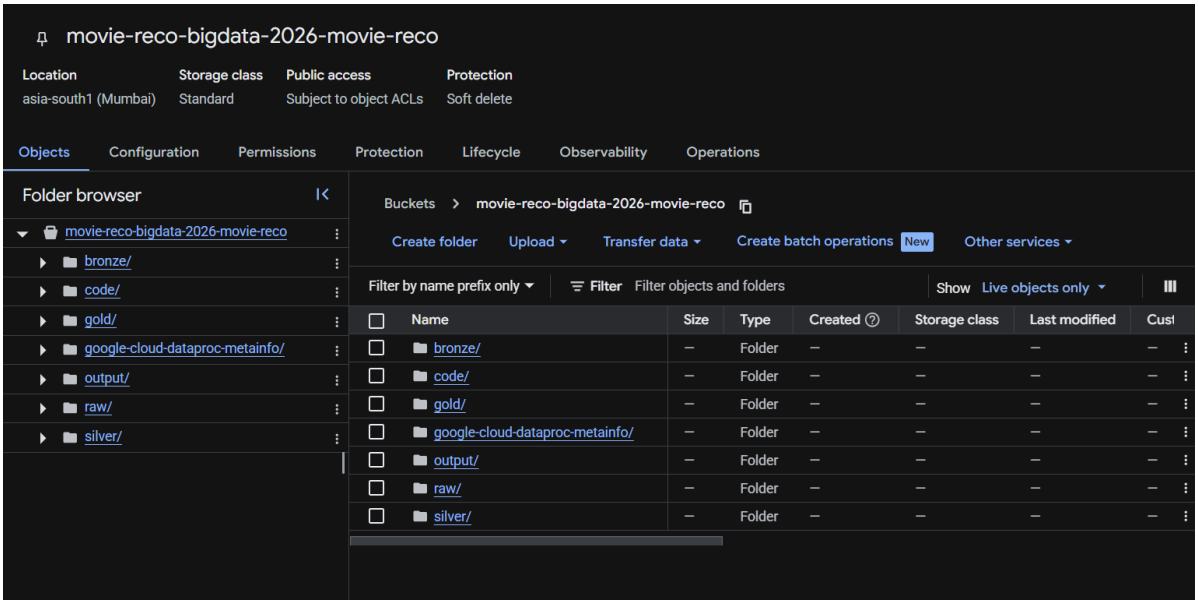


Figure 14: Google Cloud console – GCS bucket medallion layers

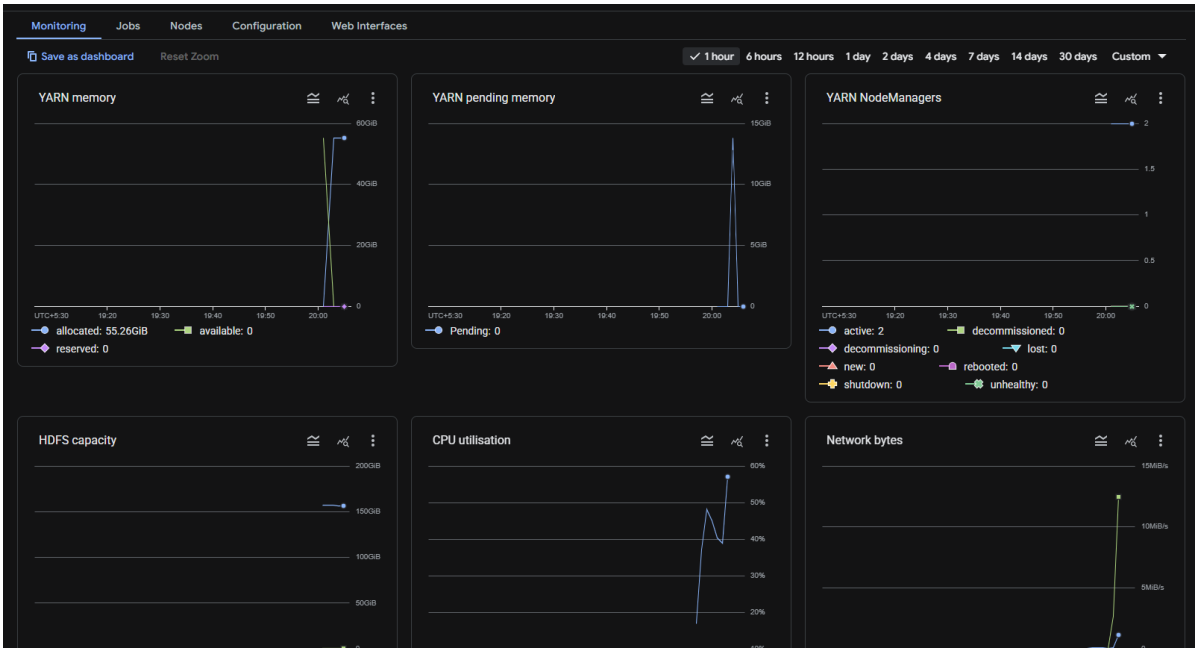


Figure 15: Google Cloud console – Dataproc cluster monitoring

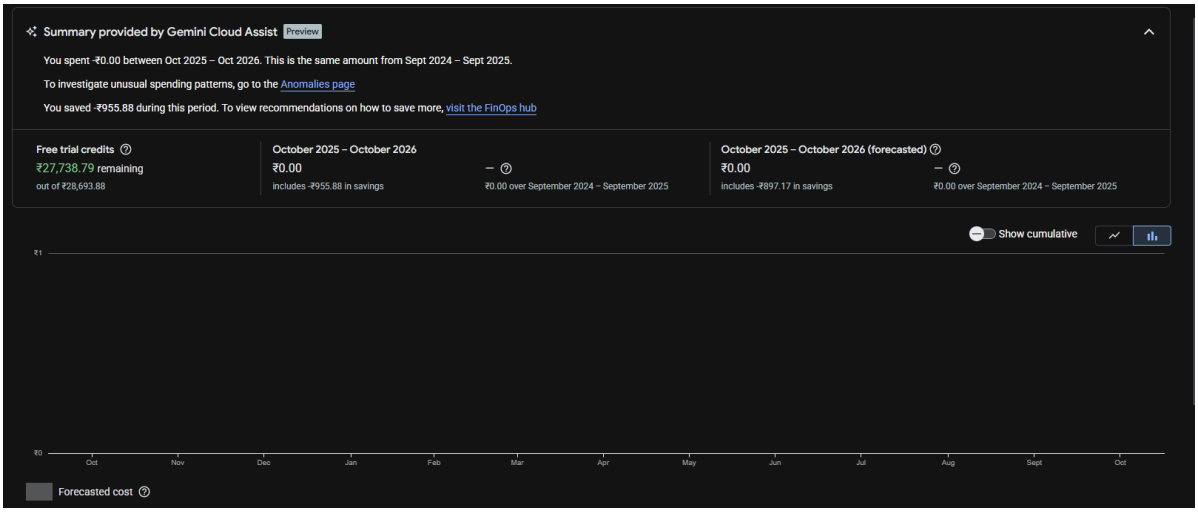


Figure 16: Google Cloud console – Billing free trial credits

Development followed a Git feature-branch workflow: each part (setup/ingest, Bronze/Silver, Gold/ML, cloud orchestration, dashboard/docs) was developed on its own branch and merged through a pull request.

8. Conclusion & Future Work

The project demonstrates scalable Big Data processing of 25,000,095 ratings on Google Cloud: a governed medallion data lake, Spark SQL analytics, a tuned MLlib recommender that clearly beats the baseline, and an interactive dashboard on real outputs. Future work: implicit-feedback ALS and hybrid content features (genres, tags) for cold-start movies, larger hyper-parameter search with CrossValidator, Spark Structured Streaming for real-time rating ingestion, and serving recommendations through an API.

References

- F. M. Harper and J. A. Konstan. The MovieLens Datasets: History and Context. ACM TiiS 5(4), 2015.
- Y. Koren, R. Bell, C. Volinsky. Matrix Factorization Techniques for Recommender Systems. IEEE Computer, 2009.
- Y. Hu, Y. Koren, C. Volinsky. Collaborative Filtering for Implicit Feedback Datasets. ICDM, 2008.
- Apache Spark 3.5 MLlib – Collaborative Filtering: https://spark.apache.org/docs/3.5.3/ml-collaborative-filtering.html
- Google Cloud Dataproc documentation: https://cloud.google.com/dataproc/docs
- GroupLens MovieLens 25M: https://grouplens.org/datasets/movielens/25m/